Programování 2

4. cvičení, 10-3-2026

Farní oznamy

1. Tento text a kódy ke cvičení najdete v repozitáří cvičení na https://github.com/PKvasnick/Programovani-2.
2. **Domácí úkoly**:
    - Domácí úkoly pro ttento týden měli různou obtížnost:
        - Přesmyčky - poměrně lehké, chtělo nápad
        - Polohy prvků ve sloučeném seznamu - většina našla správné řešení, někteří k tomu ještě hezké.
    - Pokud máte pocit, že vám programování nejde, ozvěte se. Bude hůř.

Dnešní program:

- Kvíz
- Domácí úkoly
- Matice
- Reklamy u dálnice a podobné záhady

Na zahřátí

> *"Code is like humor. When you have to explain it, it's bad." – Cory House*

Dobrý kód nepotřebuje mnoho komentářů (ale někdy potřebuje).

```
def sortedfind(x,data):
    #najde index nějakého prvku s danou hodnotou v seznamu
    #postupně osekávám seznam zleva a zprava
    minimum=data[0]#hodnota levého okraje oříznutého seznamu
    maximum=data[-1]#pravý okraj
    minindex=0#pozice levého okraje
    maxindex=len(data)-1#pravý okraj
    if minimum==x:#hledaný prvek je na začátku seznamu
        return(minindex)
    elif maximum==x:#je na konci
        return(maxindex)
    ...
```

Komentáře neudělají ze špatného kódu dobrý kód. (V tomto případě vzniklo něco jako plísňový sýr.)

Co dělá tento kód

```
d = dict.fromkeys(range(10), [])
for k in d:
    d[k].append(k)
d
```

`dict.fromkeys` vytváří z kolekce klíče slovníku a případně jim přiřadí hodnotu. Je oto někdy dobrá náhrada `defaultdict` nebo `Counter`. Tato metoda se často používá off-label namísto množiny.

Tato konkrétní inicializace slovníku je ale velmi špatný nápad. Ví někdo proč, a jak to udělat lépe?

Co dělá tento kód 2

```
from collections import defaultdict

sl = defaultdict(lambda: "Tady nic není!")

sl[1] = "Jo, tady něco je!"

sl
Out[1]: defaultdict(<function __main__.<lambda>()>, {1: 'Jo, tady něco je!'})

sl[1]
Out[2]: 'Jo, tady něco je!'

sl[2]
Out[2]: 'Nic tady není!'
```

Parametr konstruktoru `defaultdict` je *funkce*, ne název typu. Tedy když deklarujete `slovnik = defaultdict(list)`, znamená to, že pro neexistující klíč se vytvoří hodnota `slovnik[klic] = list()`, tedy podle očekávání prázdný seznam. Porovnejte s voláním `dict.fromkeys`, kde druhý parametr je konstanta, např. `[]`, a to způsobuje problémy.

Počítání věcí v Pythonu

- `dict`
- `collections.defaultdict`
- `collections.Counter`

V principu se bez speciálních udělátek obejdeme, ale jejich použití lépe signalizuje záměr v kódu a tedy jeho čitelnost.

Například toto je poctivé řešení domácího úkolu o přesmyčkách:

```
def sorted_string(string : str) -> str:
    return "".join(sorted(string))

sablony = {}
n_sablon = int(input())
for i in range(n_sablon):
```

```
    sablona = input().strip()
    sablony.setdefault(sorted_string(sablona), []).append(sablona)
n_dotazu = int(input())
for i in range(n_dotazu):
    dotaz = input().strip()
    print(*sorted(sablony.get(sorted_string(dotaz), [])))
```

- `dict.get(key, default)` Vrátí hodnotu `slovník[key]`, pokud se ve slovníku nachází, Pokud ne, vrátí hodnotu `default`.
- `dict.setdefault(key, default)` Vrátí hodnotu `slovník[key]`, pokud se ve slovníku nachází, Pokud ne, **zřídí** novou položku `key : default` a vrátí ji.

Domácí úkoly

1. ChatGPT a domácí úkol o anagramech

Chtěl jsem si vyzkoušet, jak ChatGPT zvládne vyřešit jeden z domácích úkolů. Následující je záznam z chatu:

> Prosím, vygeneruj kód v jazyce Python, který bude řešit následující úlohu:

Samozřejmě, rád vám s tím pomohu! Prosím, popište mi co nejpodrobněji, co by váš kód měl dělat, abych vám mohl připravit co nejlepší řešení.

> Na vstupu je číslo N, pak N řádků se slovy slovníku, pak číslo M a nakonec M řádků s dotazy (slovy). Pro každý dotaz najděte ve slovníku všechny jeho přesmyčky, tedy slova, která jsou složena ze stejných písmen, jen možná v jiném pořadí (každé slovo je tedy svou vlastní přesmyčkou). Přesmyčky ke každému dotazu vypište na samostatný řádek v abecedním (lexikografickém) pořadí, oddělené mezerami. Pokud k dotazu není nalezena žádná přesmyčka, vypište prázdný řádek.

Jistě, zde je kód v jazyce Python, který vyhovuje vašim požadavkům:

python

```python
from collections import Counter

# načtení vstupních hodnot
n = int(input())
dictionary = [input().strip() for _ in range(n)]
m = int(input())
queries = [input().strip() for _ in range(m)]

# vytvoření seznamu přesmyček pro každý dotaz
anagrams = {}
for query in queries:
```

```
    query_counts = Counter(query)
    anagram_list = [word for word in dictionary if Counter(word) == query_counts]
    anagram_list.sort()
    anagrams[query] = anagram_list

# vypsání výsledků
for query in queries:
    for anagram in anagrams[query]:
        print(anagram, end=' ')
    print()
```

Kód nejprve načte vstupní hodnoty a uloží je do proměnných `n`, `dictionary`, `m` a `queries`. Poté vytvoří seznam přesmyček pro každý dotaz a uloží je do slovníku `anagrams`, kde klíče jsou dotazy a hodnoty jsou seznamy přesmyček odpovídajících dotazu. Seznam přesmyček se vytvoří tak, že se pro každé slovo ze slovníku spočítají jeho písmena a porovnají se s písmeny dotazu. Pokud se shodují, slovo je přidáno do seznamu přesmyček. Seznam je seřazen lexikograficky.

Nakonec kód projde všechny dotazy a vypíše seznam přesmyček pro každý dotaz. Pokud nebyly nalezeny žádné přesmyčky, vypíše se prázdný řádek.

> Toto je dost pomalý kód, protože pro každý dotaz musí projít celým seznamem slov a u každého slova porovnat písmena, z nichž pozůstává. Nešlo by to vyhledávání implementovat efektivněji?

Ano, máte pravdu, že tento kód může být pomalý pro velká vstupní data. Efektivnější řešení by mohlo být použít slovník, kde klíčem by byla hash hodnota převedeného slova a hodnotou by byl seznam všech slov se stejnou hash hodnotou. Potom by stačilo porovnávat hash hodnoty namísto písmen pro každé slovo v seznamu, což by mělo být mnohem rychlejší.

Zde je kód s tímto vylepšením:

python

```
from collections import Counter

# načtení vstupních hodnot
n = int(input())
dictionary = [input().strip() for _ in range(n)]
m = int(input())
queries = [input().strip() for _ in range(m)]

# vytvoření slovníku hash hodnot a seznamů přesmyček pro každý dotaz
anagrams = {}
for word in dictionary:
    word_hash = hash(frozenset(Counter(word).items()))
    if word_hash in anagrams:
        anagrams[word_hash].append(word)
    else:
        anagrams[word_hash] = [word]

# vypsání výsledků
for query in queries:
```

```
    query_hash = hash(frozenset(Counter(query).items()))
    if query_hash in anagrams:
        anagram_list = sorted(anagrams[query_hash])
        print(' '.join(anagram_list))
    else:
        print()
```

Kód funguje téměř stejně jako původní kód, ale namísto porovnávání písmen pro každé slovo se používá hash hodnota vytvořená z počtu výskytů jednotlivých písmen v daném slově pomocí třídy `Counter`. Slovník `anagrams` obsahuje seznamy přesmyček pro každou hash hodnotu, což by mělo být rychlejší než hledání přesmyček v celém seznamu slov.

Pozor, tato implementace má ale jednu výraznou nevýhodu. Pokud nějaká dva různá slova mají stejnou hash hodnotu, budou přidána do stejného seznamu přesmyček, i když ve skutečnosti nejsou přesmyčkami. Tento problém lze řešit pomocí nějakého dalšího mechanismu na kontrolu, zda jsou slova skutečně přesmyčkami.

> Abychom předešli konfliktům hash hodnot, nemohli bychom namísto hashe vytvořit pro každé slovo nové slovo tak, že uspořádáme písmena slova podle abecedy? Takovéto abecedně setříděné slovo bychom pak mohli používat namísto hashe. Co si o tom myslíš?

Ano, použití abecedně setříděných slov místo hashovací funkce je další způsob, jak řešit konflikty hash hodnot, a mělo by to fungovat dobře. Toto řešení je sice jednodušší, ale může být náročnější na paměť, protože pro každé slovo musíme uložit jeho abecedně setříděnou verzi a porovnávat ji s abecedně setříděnými verzemi v seznamu při vyhledávání.

Zde je kód, který používá abecedně setříděné slovo namísto hashe:

python

```python
# načtení vstupních hodnot
n = int(input())
dictionary = [input().strip() for _ in range(n)]
m = int(input())
queries = [input().strip() for _ in range(m)]

# vytvoření slovníku přesmyček pro každý dotaz
anagrams = {}
for word in dictionary:
    sorted_word = ''.join(sorted(word))
    if sorted_word in anagrams:
        anagrams[sorted_word].append(word)
    else:
        anagrams[sorted_word] = [word]

# vypsání výsledků
for query in queries:
    sorted_query = ''.join(sorted(query))
    if sorted_query in anagrams:
        anagram_list = sorted(anagrams[sorted_query])
        print(' '.join(anagram_list))
    else:
```

```
        print()
```

Tento kód funguje podobně jako předchozí řešení s hashovací funkcí, ale namísto hashovací funkce používá abecedně setříděné verze slov. Pokud jsou dvě slova přesmyčky, budou mít stejnou abecedně setříděnou verzi a budou přidána do stejného seznamu přesmyček. Pokud tedy máme slova "abcd" a "dcba", abecedně setříděné verze obou slov budou "abcd", a proto budou přidána do stejného seznamu přesmyček.

Obě řešení mají své výhody a nevýhody a vhodné řešení závisí na konkrétní situaci. Pokud jsou slova vstupu relativně krátká, použití abecedně setříděných slov může být efektivnější. Na druhé straně, pokud jsou slova delší a mají velký počet různých znaků, může použití hashovací funkce být rychlejší a méně náročné na paměť.

-- konec GPT chatu --

Co si o tom myslíte?

Pozice prvků ve sloučené posloupnosti

Tímto se nebudu zabývat, většinou jste našli dobré řešení založené na algoritmu slučování posloupností:

```
def read_from_console() -> tuple[list[int], list[int]]:
    n, m = [int(s) for s in input().split()]
    a = [int(s) for s in input().split()]
    assert len(a) == n, "Error in reading from console."
    b = [int(s) for s in input().split()]
    assert len(b) == m, "Error in reading from console."
    return a, b


def solve(a: list[int], b: list[int]) -> tuple[list[int], list[int]]:
    a_indices = []
    b_indices = []
    i = 0
    j = 0
    k = 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            a_indices.append(k + 1)
            i += 1
        else:
            b_indices.append(k + 1)
            j += 1
        k += 1
    while i < len(a):
        a_indices.append(k + 1)
        i += 1
        k += 1
    while j < len(b):
        b_indices.append(k + 1)
        j += 1
        k += 1
    return a_indices, b_indices
```

```python
def main() -> None:
    a, b = read_from_console()
    a_indices, b_indices = solve(a, b)
    print(*a_indices)
    print(*b_indices)


if __name__ == "__main__":
    main()
```

Vyskytlo se několik zajímavých variací; tato pochází od kolegy Jáchyma Galušky:

```python
n,m = map(int,input().split())
z = [int(x) for x in input().split()]
g = [int(x) for x in input().split()]

l = 0
indexy1 = []
for i in range(n):
    while l < m:
        if z[i] > g[l]:
            l += 1
        else:
            break
    indexy1.append(l + i + 1)

indexy2 = []
j=0
for i in range(1, n + m + 1):
    if j < len(indexy1) and indexy1[j] == i:
        j += 1
    else:
        indexy2.append(i)

print(' '.join(map(str, indexy1)))
print(' '.join(map(str, indexy2)))
```

Co u této úlohy nemůžete udělat, je spojit vstupní posloupnosti a setřídit je. Pokud si ale u každé hodnoty označíte, odkud pochází, je to docela funkční:

```
m, n = [int(s) for s in input().split()]
a = [int(s) for s in input().split()]
b = [int(s) for s in input().split()]

tuples = [(ai, "A") for ai in a] + [(bi, "B") for bi in b]
tuples.sort()   # proč tady nemáme parametr key= ?
a_index = [i for i, (val, flag) in enumerate(tuples) if flag == "A"]
b_index = [i for i, (val, flag) in enumerate(tuples) if flag == "B"]

print(*a_index)
print(*b_index)
```

Matice

Samotný Python se nehodí pro numerické počítání, ale máme k dispozici modul `numpy` a ekosystém modulů na něj navazující, které umožňují efektivní výpočty, včetně operací s maticemi a lineární algebry.

Problém s maticemi je zhruba v tom, že násobení dvou matic $n \times n$ má složitost $O(n^3)$. Tedy pokud chceme počítat s velkými maticemi, musíme si výpočty precizně zorganizovat a použít prostředky, které naplno využijí možnosti počítače. A Python jako interpretovaný jazyk s dynamickým typováním není na tento účel příliš vhodný.

My si dnes uděláme třídu `Matrix`, a zkusíme implementovat základní i podivné operace s maticemi. Váš domácí úkol bude využít tuto třídu a udělat další operaci.

```
# This module implements a matrix class, with matrix data stored in a flat list.
# We start by implementing basic matrix operations.


class Matrix:
    def __init__(self, rows, cols, data = None):
        self.rows = rows
        self.cols = cols
        if data is None:
            self.data = [0] * (rows * cols)
        else:
            self.data = data.copy()

    def __getitem__(self, index):
        row, col = index
        return self.data[row * self.cols + col]

    def __setitem__(self, index, value):
        row, col = index
        self.data[row * self.cols + col] = value

    def __str__(self):
        strmat = []
        for row in range(self.rows):
```

```
            strmat.append(
                " ".join(map(str, self.data[row * self.cols : (row + 1) * self.cols]))
            )
        return "\n".join(strmat)

    def __repr__(self):
        return f"Matrix object with dimensions {self.rows}x{self.cols}: " + repr(
            self.data
        )

    def __add__(self, other):
        if self.rows != other.rows or self.cols != other.cols:
            raise ValueError("Matrices must have the same dimensions")
        result = Matrix(self.rows, self.cols)
        for i in range(self.rows * self.cols):
            result.data[i] = self.data[i] + other.data[i]
        return result

    def __sub__(self, other):
        if self.rows != other.rows or self.cols != other.cols:
            raise ValueError("Matrices must have the same dimensions")
        result = Matrix(self.rows, self.cols)
        for i in range(self.rows * self.cols):
            result.data[i] = self.data[i] - other.data[i]
        return result

    def __mul__(self, other):
        if self.cols != other.rows:
            raise ValueError("Matrices must have compatible dimensions")
        result = Matrix(self.rows, other.cols)
        for i in range(self.rows):
            for j in range(other.cols):
                result[i, j] = sum(self[i, k] * other[k, j] for k in range(self.cols))
        return result

    def __eq__(self, other):
        if self.rows != other.rows or self.cols != other.cols:
            return False
        return all(
            self[i, j] == other[i, j]
            for i in range(self.rows)
            for j in range(self.cols)
        )

    def reshape(self, rows, cols):
        if rows * cols != self.rows * self.cols:
            raise ValueError("Cannot reshape matrix to the given dimensions")
        result = Matrix(rows, cols)
        result.data = self.data.copy()
        return result

    def vec(self):
        return self.reshape(self.rows * self.cols, 1)
```

```
    def flatten(self):
        return self.reshape(1, self.rows * self.cols)

    def transpose(self):
        result = Matrix(self.cols, self.rows)
        result.data = [
            self.data[j * self.cols + i]
            for i in range(self.cols)
            for j in range(self.rows)
        ]
        return result


def main():
    m = Matrix(3, 4)
    for i in range(m.rows):
        for j in range(m.cols):
            m[i, j] = i + j
    print(m)
    print(m + m)
    print(m.vec())
    print(m.flatten())
    print(m.transpose())
    print(m * m.transpose())


if __name__ == "__main__":
    main()
```

Můžeme se ptát, nač je dobré implementovat matici jako plochý seznam. Vidíte, že to pomáhá u operací sčítání a odečítání, a také u operací, spojených se změnou tvaru matice (reshape). Na druhé straně u násobení velký přínos nevidíme.

Reklama u dálnice a jiné podivnosti

Od Dr. Forstové jste dostali za úlohu najít největší mezeru v datech, tedy najít největší rozdíl mezi sousedícími prvky v setříděné posloupnosti, a udělat to tak, abyste posloupnost nemuseli celou setřídit.

Nedostanete kompletní řešení, jenom několik poznámek.

1. Triviální řešení (O(n log n))

```
from itertools import pairwise
from random import randint

data = [randint(0, 100) for _ in range(20)]

data.sort()
max_spacing = max(q - p for p, q in pairwise(data))
print(*data)
print(max_spacing)
```

2. Podobná úloha: Jak najít nejbližší prvky posloupnosti?

Jak se tato úloha liší od původní a co mají společné?

Klíč k řešení obou úloh je uvědomit si,

- jaká je minimální maximální vzdálenost mezi sousedními prvky
- Jaká je maximální minimální vzdálenost mezi sousedními prvky

a jak (a jestli) můžeme tento vhled použít ke "zmenšení" té-které úlohy.

Třídění

Abychom mohli věci třídit, musí tyto věci implementovat operátory porovnání:

Nejprve si prosvištíme pár jednoduchých metod:

Selection sort

Zleva nahrazujeme hodnoty minimem zbývající části posloupnosti:

```
def selection_sort(b):
    for i in range(len(b) -1):
        j = b.index(min(b[i:]))
        b[i], b[j] = b[j], b[i]
    return b
```

$O(n^2)$ operací, $O(1)$ paměť.

Bubble sort

```
def bubble_sort(b):
    for i in range(len(b)):
        n_swaps = 0
        for j in range(len(b)-i-1):
            if b[j] > b[j+1]:
                b[j], b[j+1] = b[j+1], b[j]
                n_swaps += 1
        if n_swaps == 0:
            break
    return b
```

$O(n^2)$ operací, $O(1)$ paměť.

Insertion sort

Začínáme třídit z kraje seznamu, následující číslo vždy zařadíme na správné místo do již utříděné části.

```
def insertion_sort(b):
    for i in range(1, len(b)):
        up = b[i]
        j = i - 1
        while j >= 0 and b[j] > up:
            b[j + 1] = b[j]
            j -= 1
        b[j + 1] = up
    return b
```

$O(n^2)$ operací, $O(1)$ paměť.

a teď něco, co je opravdu efektivní:

Bucket sort

- Nahrubo si setřídíme čísla do příhrádek
- Setřídíme obsah přihrádek
- Spojíme do výsledného seznamu

```
def bucketSort(x):
    arr = []
    slot_num = 10 # 10 means 10 slots, each
                  # slot's size is 0.1
    for i in range(slot_num):
        arr.append([])

    # Put array elements in different buckets
    for j in x:
        index_b = int(slot_num * j)
        arr[index_b].append(j)

    # Sort individual buckets
    for i in range(slot_num):
        arr[i] = insertionSort(arr[i])

    # concatenate the result
    k = 0
    for i in range(slot_num):
        for j in range(len(arr[i])):
            x[k] = arr[i][j]
            k += 1
    return x
```

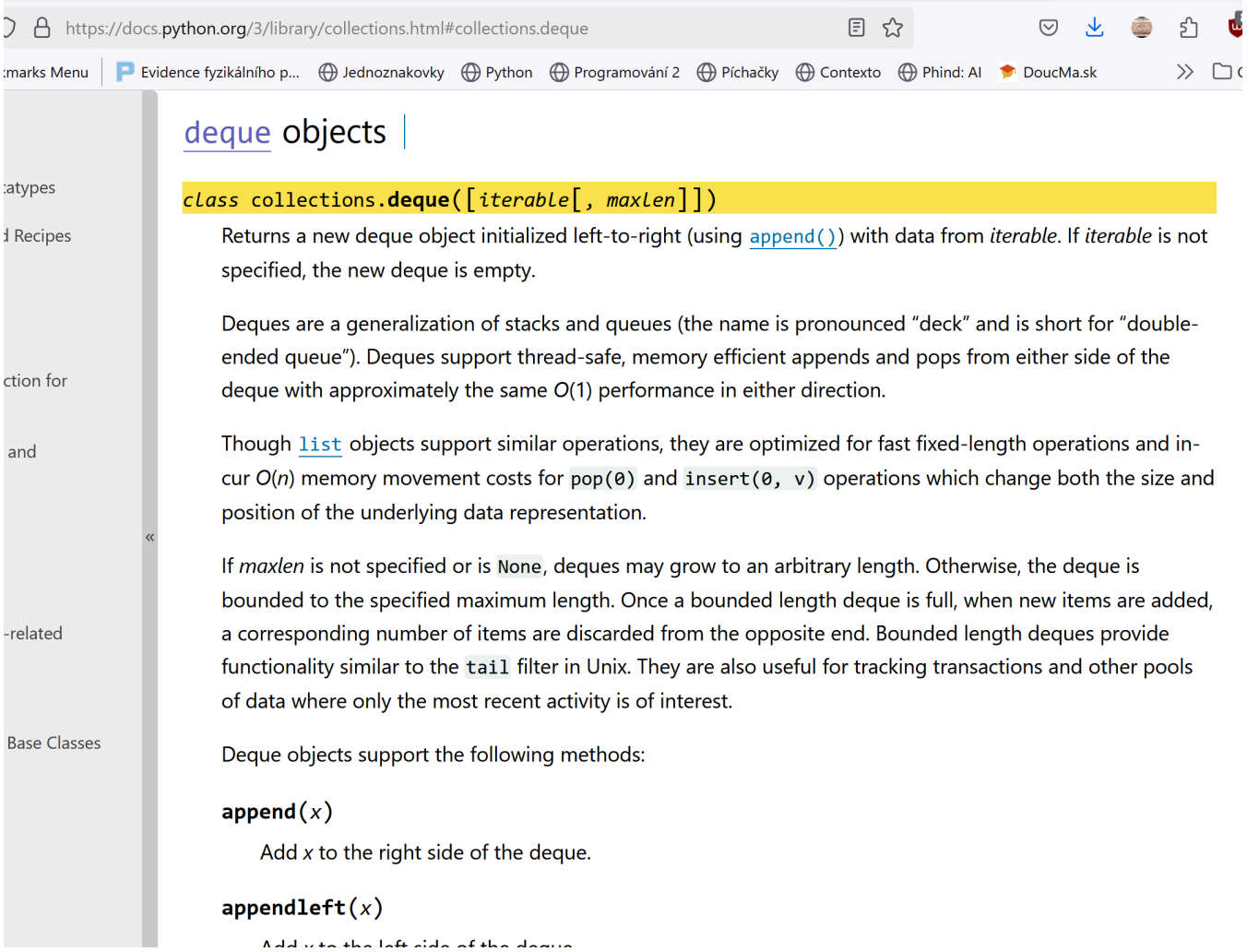
To je docela ošklivý kód, uměli bychom ho vylepšit?

Když ale obsah přihrádek nemusíme třídit, dostáváme třídění s lineární složitostí.

Také můžeme uvnitř přihrádek použít stejný princip a dělit rekurzivně podle dalších (jemnějších) kritérií. (radix sort).

collections.deque - pole optimalizované pro přídávání a odebírání prvků na začátku a na konci.

!



`pop`, `popleft`, `append`, `appendleft`.

Částečné třídění, quickselect a quicksort

Ještě se na chvíli vrátíme k částečnému třídění pomocí pivotování. Algoritmus můžeme symbolicky brát jako bucket sort, přičemž používáme tři buckety.

Např. symbolická implementace QuickSortu může vypadat nějak takto:

```
from random import randint

def quicksort(data):
    if len(data) <= 1:
        return data
    if len(data) == 2:
        return [min(data), max(data)]
```

```
    pivot = data[randint(0, len(data)-1)]
    left = [i for i in data if i < pivot]
    pivots = [i for i in data if i == pivot]
    right = [i for i in data if i > pivot]
    return quicksort(left) + pivots + quicksort(right)


data = [randint(1,100) for _ in range(20)]
print(data)
sdata = quicksort(data)
print(sdata)
```

Pro QuickSelect pokračujeme pouze s částí, kde se nachází požadovaný index.

To ale není realistický algoritmus, protože potřebuje mnoho paměti a skládá seznamyy atd. Umíme rozdělit data do tří skupin na místě?

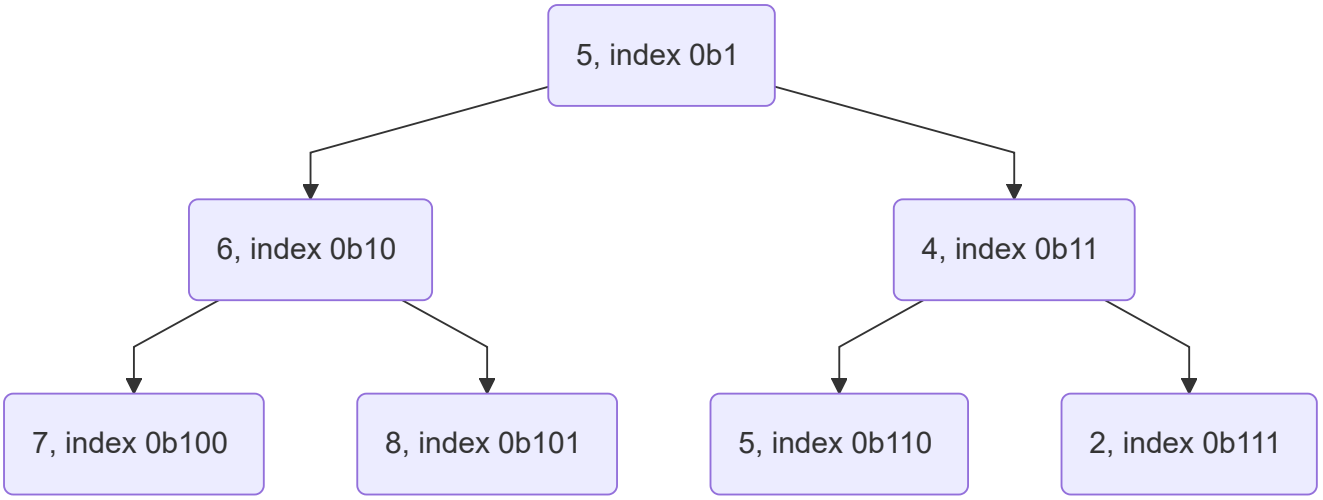
Jak podle vás funguje tento kód? Všimněte si, jak vám v čtení pomáhají komentáře. Jaká je složitost tohoto kódu?

```
def partition(data, left, right, pivotIndex, n):
    pivotValue = data[pivotIndex]
    data[pivotIndex], data[right] = data[right], data[pivotIndex]  # Move pivot to end
    storeIndex = left
    # Move all elements smaller than the pivot to the left of the pivot
    for i in range(left, right):
        if data[i] < pivotValue:
            data[storeIndex], data[i] = data[i], data[storeIndex]
            storeIndex += 1
    # Move all elements equal to the pivot right after
    # the smaller elements
    storeIndexEq = storeIndex
    for i in range(storeIndex, right):
        if data[i] == pivotValue:
            data[storeIndexEq], data[i] = data[i], data[storeIndexEq]
            storeIndexEq += 1
    data[right], data[storeIndexEq] = data[storeIndexEq], data[right]  # Move pivot to its
final place
    # Return location of pivot considering the desired location n
    if n < storeIndex:
        return storeIndex  # n is in the group of smaller elements
    if n <= storeIndexEq:
        return n  # n is in the group equal to pivot
    return storeIndexEq  # n is in the group of larger elements

data = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
print(data)
pivotIndex = 2  # 3rd element is 4
n = 6        # 6th smallest element is 6
result = partition(data, 0, len(data)-1, pivotIndex, n)
print("Partitioned index:", result)
print("Data after partitioning:", data)
```

Halda - heap

Binární strom, implementovaný v seznamu. Namísto struktury stromu používámee vztahy přes indexy:



- Potomci uzlu na indexu k jsou 2k a 2k+1
- Předek uzlu na indexu k je k // 2
- Uzel k je levý potomek svého předka, pokud k % 2 == 0, jinak je to pravý potomek.

Pravidlo min-haldy (min-heap): potomci uzlu jsou větší než hodnota v uzlu.

```
# heap implementation
from random import randint

def add(h:list[int], x:int) -> None:
    """Add x to the heap"""
    h.append(x)
    j = len(h)-1
    while j > 1 and h[j] < h[j//2]:
        h[j], h[j//2] = h[j//2], h[j]
        j //= 2


def pop_min(h: list[int]) -> int:
    """remove minimum element from the heap"""
    if len(h) == 1: # empty heap
        return None
    result = h[1]   # we have the value, but have to tidy up
    h[1] = h.pop()  # pop the last value and find a place for it
    j = 1
    while 2*j < len(h):
        n = 2 * j
        if n < len(h) - 1:
            if h[n + 1] < h[n]:
                n += 1
```

```
        if h[j] > h[n]:
            h[j], h[n] = h[n], h[j]
            j = n
        else:
            break
    return result


def main() -> None:
    heap = [None]  # no use for element 0
    for i in range(10):
        add(heap, randint(1, 100))
        print(heap)
    for i in range(len(heap)):
        print(pop_min(heap))
        print(heap)


if __name__ == '__main__':
    main()
```

Domácí úkoly

1. **Bucket sort pro seznam slov:** Máte pomocí bucket sort (nebo spíš radix sort, protože se žádá rekurzivní třídění bucketů) setřídit seznam slov. Musíte si poradit se slovy různé délky v souladu s tím, jak se slova různé délky třídí podle abecedy.
2. **Permutace řádků matice** - máte změnit pořadí řádků matice podle zadaného permutačního vektoru. Použijte implementaci, kterou jsme dnes probírali na cvičení.